

Master's programme in Computer, Communication and Information Sciences

Assessing Viability of Hierarchical Reinforcement Learning in Video Game Development

Saku Komulainen

© 2024

This work is licensed under a [Creative Commons](#)
“Attribution-NonCommercial-ShareAlike 4.0 International” license.



Author Saku Komulainen

Title Assessing Viability of Hierarchical Reinforcement Learning in Video Game Development

Degree programme Computer, Communication and Information Sciences

Major Machine Learning, Data Science and Artificial Intelligence

Supervisor Senior University Lecturer Vesa Hirvisalo

Advisor M.Sc. Anton Debner

Date 19th of September 2024 **Number of pages** 46 **Language** English

Abstract

In this thesis we explore the feasibility of using hierarchical reinforcement learning (HRL) in video game development to create non-player characters (NPC). NPCs are a crucial part of video games affecting many parts of the game, including storytelling, atmosphere, and importantly work as opponents and teammates. Using traditional methods to create NPCs in video games can be a lengthy and difficult process requiring expert knowledge. Reinforcement learning (RL) has shown potential, but has remained largely unused in video game development due to some major issues. HRL provides solutions to these issues, allowing the complex task to be split into smaller, easier to learn sub-tasks.

We design, implement, and study a new HRL method with the potential of creating NPCs with multiple competency levels with minimal effort. Our design is based on a goal-conditional framework which we modify to suit our goals. Instead of using a goal-vector we repurpose it to a skill-vector, which could allow us to mask it and re-train the higher-level policy to prevent certain skills from being used. In order to experiment with our HRL method, we create an physics based quadruped locomotion environment that has possibility for learning multiple different skills. We evaluate our method with and without information hiding in attempt to force certain types of behaviours for the policy levels. The method shows potential in our experiments but requires further experimentation and engineering to create multiple competency levels.

Keywords Computer Science, Video Game Development, Reinforcement Learning, Hierarchical Reinforcement Learning, HRL, Deep Learning, Ray RLLib, Unity

Tekijä Saku Komulainen

Työn nimi Hierarkkisen Vahvistusoppimisen Soveltuvuuden Arviointi Videopelien Kehittämisessä

Koulutusohjelma Computer, Communication and Information Sciences

Pääaine Machine Learning, Data Science and Artificial Intelligence

Työn valvoja Vanhempi yliopistonlehtori Vesa Hirvisalo

Työn ohjaaja DI Anton Debner

Päivämäärä 19.9.2024

Sivumäärä 46

Kieli englanti

Tiivistelmä

Diplomityössä on tutkittu hierarkkisen vahvistusoppimisen (hierarchical reinforcement learning, HRL) käytön soveltuvuutta videopelien kehityksessä ei-pelaaja-hahmojen (non-player character, NPC) luomiseen. NPC-hahmot ovat tärkeä osa videopelien tarinankerrontaa, tunnelmaa ja ne myös toimivat vastustajina ja joukkuetovereina. Perinteisten kehitysmenetelmien käyttäminen NPC-hahmojen luomisessa saattaa viedä huomattavasti aikaa ja vaatia merkittävän määrän asiantuntemusta kyseisestä pelistä. Videopelien kehityksessä vahvistusoppiminen (reinforcement learning, RL) on osoittanut potentiaalia NPC-hahmojen luomisessa, mutta sen käyttö on yleisesti jäänyt vähäiseksi merkittävien ongelmien vuoksi. HRL-menetelmät tarjoavat ratkaisuja näihin ongelmiin pilkkomalla monimutkaiset tehtävät moneen pienempiin helpommin opittaviin osatehtäviin.

Työssä on suunniteltu, toteutettu ja tutkittu uutta HRL-menetelmää, jolla on potentiaalia luoda monia NPC-hahmoja eri osaamistasoilla pienten muokkausten avulla. Tutkimuksen HRL-menetelmä pohjautuu goal-conditional -kehikseen, jota muokattiin työn tavoitteisiin sopivaksi. Kehyksen tavoitevektoria käytettiin eri tavalla, jolloin se toimii taitovektorina. Taitovektorin avulla voidaan mahdollisesti estää NPC:tä käyttämästä tiettyjä taitoja, ja se mahdollistaisi eri osaamistasojen luomisen. Työssä luodaan fysiikkaan pohjautuva ympäristö neliraajaisella olennolla, jotta HRL-menetelmää voidaan kokeilla ympäristössä, jossa on mahdollisuus oppia monia taitoja. Kehitettyä menetelmää arvioidaan kahdella tapaa, käyttämällä tiedon piilottamista ja ilman. Tiedon piilottamisen tavoitteena on pakottaa tietäntyyppisiä käyttäytymistapoja ja taitoja eri hierarkkian tasoille. Kokeilujen perusteella kehitetyllä menetelmällä on potentiaalia luoda eri osaamistasoja, mutta se vaatii lisää kehitystyötä ja kokeiluja.

Avainsanat Tietotekniikka, Videopelien Kehitys, Vahvistusoppiminen,
Hierarkkinen Vahvistusoppiminen, Syväoppiminen, Ray RLlib, Unity

Preface

Firstly, I would like to thank my supervisor Dr. Vesa Hirvisalo and my advisor M.Sc. Anton Deber and the other members of the research team. Their support, advice, and ideas were crucial during the experiment and writing process. Secondly, I would like to thank my family for their continued encouragement and support during my studies especially while writing the masters thesis.

Espoo, 19th of September 2024

Saku Komulainen

Contents

Abstract	3
Abstract (in Finnish)	4
Preface	5
Contents	6
1 Introduction	7
2 Background	9
2.1 NPCs in video games	9
2.2 Deep Learning	10
2.3 Reinforcement Learning	11
2.4 Deep Reinforcement Learning	13
2.5 Hierarchical Reinforcement Learning	13
2.6 HRL Frameworks	14
2.7 Game Engines and RL environments	16
3 Methods	18
3.1 Base HRL Method	18
3.2 Our HRL Design	19
4 Implementation	22
4.1 Implementation of our HRL method	22
4.2 Interfacing Ray RLlib with Unity Game Engine	24
4.3 Issues with Ray and RLlib	25
5 Experiment Setup	29
5.1 Environment Description and Implementation	29
5.2 Experiment Metrics	31
5.3 Experiment Details, Hyperparameters and Configuration Options	32
6 Results	35
6.1 Baseline without HRL	35
6.2 Our HRL Approach Findings	36
6.3 Ray RLlib Observations	39
7 Discussion and Future Work	41
7.1 Future Work	42
8 Conclusions	43
References	44

1 Introduction

The field of Reinforcement Learning (RL) [1] has demonstrated potential in various applications, including video game development, notably creation of non-player characters (NPCs) [2]. NPCs are a crucial part of video games; however, the traditional methods of making them often rely on hard-coding behaviour, predefined rules, and behaviour trees [2]. Video games, especially competitive ones, commonly include NPCs with multiple competency levels to provide the players with different levels of difficulties. RL offers an alternative by enabling NPCs to learn their behaviour by interacting with the environment, effectively converting man-hours into training time. Despite their versatility in a wide variety of complex environments, many traditional RL approaches can struggle within video game environments. Hierarchical reinforcement learning (HRL) [3] offers a solution to this problem by decomposing the larger tasks into multiple smaller and easier to train sub-behaviours.

Traditional RL methods face significant challenges when applied to more complex video game settings. These challenges include factors such as difficulties in learning complex and difficult to solve problems, environment exploration, credit assignment and sample efficiency [3]. In many cases, this may lead to the RL agents failing to learn appropriate behaviour and in turn failing to solve the problems. HRL addresses these challenges by abstracting the problems into multiple more manageable smaller problems or sub-tasks. It has been shown that the HRL approach can not only reduce training time by accelerating the learning process due to the easier to learn sub-tasks but also by reusing already learned sub-behaviours [3]. However, video game development widely has not made use of RL in general [2] and this is especially true for HRL.

In the context of video game development and RL research, Unity ML-Agents [4] and Ray RLlib [5] are two widely used RL frameworks. Namely, ML-Agents is known for ease of interfacing with the Unity game engine and RLlib is known for its wide range of capabilities and flexibility for different applications. Both of the frameworks come with various different RL algorithms suitable for variety of tasks, offering accessible ways to implement agents. Unity ML-Agents is integrated well with the Unity game engine, allowing for the trained agents model to be used inside the engine. Ray RLlib can also communicate with the Unity game engine, while offering more flexibility in implementing HRL methods. As neither of the tools have built in methods of implementing HRL easily, it is not clear how suitable either of them are for using HRL to create NPCs in video games.

This thesis aims to explore the viability of using HRL in video game development for creating NPCs specifically in a physics simulated environment. We also aim to assess the ease of implementation, performance and training efficiency of Ray RLlib for both HRL and standard RL methods. From these points, we can ask further questions, such as the viability of creating multiple different levels of competencies for NPCs with a minimal effort.

The author was responsible for the design of the methods described in Section 3.2 and for the implementation described in Section 4.1. In addition, the author made modifications to the RLlib-Unity interface described in Section 4.2. Lastly, the author created the obstacle environment and modified the existing Unity ML-Agents crawler

environment described in Section 5.1.

Section 2 presents some of the main background concepts used later in the thesis. The specifics of the methods used in the experiment are described in detail in Section 3. Section 4 describes the implementation of the methods in Ray RLLib, how Ray RLLib is interfaced with the Unity game engine, and addresses some of the implementation issues encountered with Ray RLLib. This is followed by details about the experiment, including description of the experiment environment, metrics and hyperparameters in Section 5. Finally, the results are shown and discussed in Section 6 and the conclusions and future work are discussed in Section 8.

2 Background

This section provides an overview of the foundational concepts used in the thesis. The first section briefly describes the role of NPCs in video games and what their creation involves on a higher level. We then explain the basic idea of deep learning with a very brief introduction to machine learning. This is followed by introducing some of the core concepts of reinforcement learning, including Markov decision process, on- and off-policy methods, and environments. We then describe the combination of deep learning and reinforcement learning. The next section focuses on further extension to reinforcement learning, introducing hierarchical structure to solve more complex problems. Lastly, we briefly take a look at software facilitating the simulation of the environments for reinforcement learning.

2.1 NPCs in video games

NPCs are a crucial element of video games for creating immersive scenarios within the game world [2]. These NPCs may be created for many purposes including storytelling, trading, and atmosphere alongside the focus of this thesis, enemies and allies. The techniques and methodologies used to create and control these NPCs are encompassed in the field of game artificial intelligence (Game AI). Game AI can often be split up to three main categories: the strategy for group of NPCs, decision making for individual actions and lastly the movement of the characters [2].

Strategy may not be necessary for some games, as it is mostly concerned about coordinating with the other (non-player) characters in the team [2]. As such, it generally looks at the bigger picture and may essentially set long-term goals for the individual NPCs for behaviours such as grouping up.

The decision making process is used for selecting an appropriate action or behaviour depending on the environment and conditions of the character [2]. There can be a wide range of complexities within the decision making processes depending on the NPC. For example, a NPC used for atmosphere might only sit at a desk at idle unless the player bumps into the NPC causing it to stand up and look at the player. On the other hand, an enemy NPC might have a large number of behaviours including moving, attacking, hiding and patrolling [2]. The strategy mentioned before may impact the decision making process, resulting in complex group behaviour.

Movement in this case mainly refers to pathfinding and it is controlled by the decision making process. In the simplest case, movement may be linear motion, but more commonly it involves moving in an complex environment with obstacles and inside buildings [2]. Additionally, a NPC may have multiple types of movement dependent on the decision, as it may patrol while walking, sneak when it detects an enemy or run if actively engaged in combat.

Some of these elements, especially movement, may work well with relatively simple algorithms such as the A* pathfinding algorithm. However, strategy and especially decision making often ends up being done using hard coded behaviour [2]. This often requires the use of experts manually designing the behaviour of the characters consuming a considerable amount of man-hours.

2.2 Deep Learning

Rather than hard-coding solutions to problems, we can use methods to have the computer learn a solution to the problem via statistical algorithms. These algorithms that learn and extract patterns from data belong to the field of Machine Learning (ML). Depending on the dataset, we may use either supervised or unsupervised learning algorithms [6]. In the unsupervised case, the dataset only contains features, which can be used to learn patterns or the probability distribution that generated the dataset. The supervised case adds a label to the dataset, which can be used to learn to predict the label from the features. There are some difficulties with the simpler algorithms, including feature selection and further, extracting higher level abstract features [6]. For example, solving the problem of detecting complex shapes with variation in the position of the camera taking the pictures.

In order to solve this problem, we can use Deep Learning (DL) to build increasingly abstract representations of the data. To achieve this, a common approach in DL is to use an Artificial Neural Network (ANN), such as the fully connected feedforward neural network seen in Figure 1. The input layer is fed with the original data and is followed by several (two in our example figure) hidden layers that extract increasingly abstract features. Lastly, some desired representation of the input data is produced at the output node. The nodes contain weights for each of the incoming connections, which are used alongside the inputs to calculate the output of the node. Additionally, the nodes are commonly followed by a non-linear activation function which allow the networks to learn non-linear patterns.

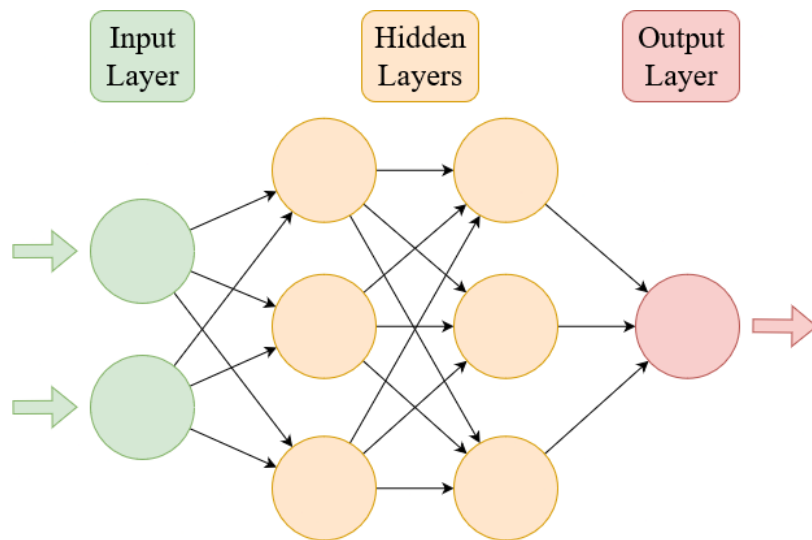


Figure 1: A fully connected feedforward network consisting of input layer with two input nodes, two hidden layers with three nodes each and a output layer with a single output node.

2.3 Reinforcement Learning

In order to solve problems in dynamic environments, such as video games, neither supervised nor unsupervised ML methods are sufficient. For supervised learning it would be impractical, and often even impossible, to collect and label datapoints from all of the possible states of the environment such that a correct or suitable action could be taken [1]. While unsupervised learning is close to what we are looking for, it does not have the correct objective as it tries to find a hidden structure [1]. Instead, we want to learn what action should be taken in a given environment state to maximize the reward signal, learned by trial-and-error.

These dynamic environments can often be formalized as finite Markov decision processes (MDP) where an agent takes actions, influencing future states and subsequently affecting the rewards given to the agent. Essentially, MDP is a mathematically idealized form of the problem of learning from interactions to achieve a goal and provides a framework for modelling decision problems [1]. MDP can be defined by a tuple containing state space S , action space A , state-transition probabilities $p(s_{t+1}|s_t, a_t)$ and rewards R . Alongside these main elements, MDP contains an environment and an agent that learns and takes actions in the environment. The main elements of MDP and their interaction with the agent and the environment can be seen in Figure 2. The environment starts in an initial state S_0 , which the agent uses to decide on the action A_0 , resulting in reward R_1 and state S_1 the next timestep, followed by the agent taking another action. This process is looped until the agent takes an action resulting in a terminal state S_T , which could include the agent reaching the goal or a failure condition such as running out of time. This process results in a trajectory containing all the states, actions and rewards in order: $S_0, A_0, R_1, S_1, \dots, R_T, S_T$ [1].

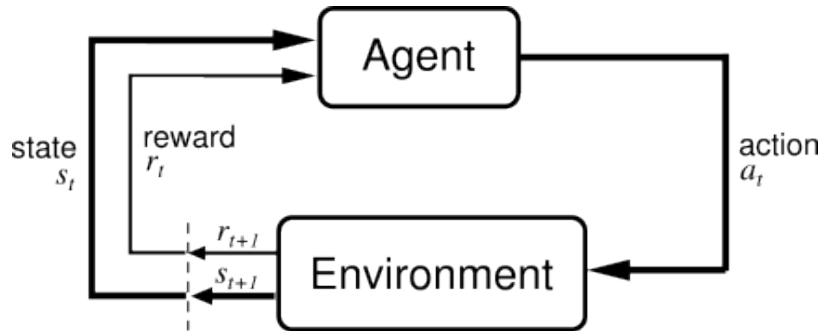


Figure 2: Depiction of MDP agent-environment interaction. Figure [1].

MDP provides a basis for RL environments and agents, however, in many cases the idealized version of MDP does not fully apply. The environment can be either fully or partially observed, with the latter case therefore being partially observable MDP [1]. In the first case, every single detail about the environment is available to the agent, including games such as checkers and chess but is often impractical or impossible in more complex environments. In the partially observable MDP only a part of the state is known, and is more common in real-world applications such as robotic arms or self-driving vehicles. In such scenario we use the term observation

rather than state to distinguish between the two cases. Another factor that poses issues to MDP is scenarios with the environment containing multiple agents, as this breaks the Markov property assuming that the environment is stationary and only reacts to a single agent's actions [1].

RL aims to maximize the cumulative reward obtained from the environment. This is achieved by learning a policy $\pi(a|s)$, which learns to take actions that maximize the expected discounted return $G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k$, where γ is the discount factor [1]. The discount factor is used to bias how much the agent takes expected future rewards into consideration when choosing the next action. Additionally, this allows for the agent to perform in a continuous task, where theoretically the expected reward could be infinite without discounting [1].

RL can be split into two categories depending on how policies are handled during the training process. These categories are on-policy and off-policy methods, which mainly differ by whether the behaviour and target policies are the same [7]. The target policy in this case refers to the desired policy we want the agent to learn and the behaviour policy is used to interact with the environment. Both of these methods have benefits and drawbacks depending on the specific problem.

In an on-policy method the behaviour policy is the same as the target policy, as such the target policy is used to generate samples from the environment. As soon as the policy optimization has been completed on the samples, they can be discarded, saving resources [7]. The main benefit of on-policy methods is the relative simplicity due to only having a single policy and the ability to directly optimize the policy [7]. Proximal policy optimization (PPO) [8] is an example of an on-policy method, which has been shown to perform well in multiple different types of environments. The implementation of PPO is relatively simple, being comparable to a vanilla policy gradient implementation while also outperforming many other on-policy methods in most of the cases [8].

On contrary to on-policy methods, off-policy methods have separate policies for training and for exploring the environment. The main benefit is being able to use past experiences from different policies for training, thus improving sample efficiency significantly [7]. However, this comes at the cost of having to store old experiences or increased complexity of deciding on what experiences should be kept. Twin delayed deep deterministic policy gradient (TD3) [9] is an example of an off-policy method, which is well suited for various applications with improved training stability compared to other methods. TD3 has been shown to outperform PPO significantly in more complex environments at the same training steps, displaying the significantly higher sample efficiency.

An important factor for training appropriately behaving agents is the choice of hyperparameters that influence how the agent is trained. There are multiple ways to tune these hyperparameters, including manual trial-and-error and automatic methods such as grid search and population based training (PBT) [10]. However, manual trial-and-error hyperparameter tuning can be time consuming and may result in sub-optimal agent behaviour and increased training time due to the large search space [11]. The automatic methods, specifically PBT has been shown to be beneficial in multiple ways, including faster training and better adaptability to changing environments [10].

2.4 Deep Reinforcement Learning

While the more traditional RL methods discussed in the section above work well in more simple cases, they often struggle in more complex environments. These environments can contain high-dimensional state-spaces and/or action-spaces that make the more traditional RL methods infeasible [7]. In such cases, we may use Deep Reinforcement Learning (DRL) where we approximate the policy $\pi(a|s)$, the value function and/or other relevant functions with ANNs. The on- and off-policy methods, PPO and TD3, mentioned in the previous sections commonly are examples of DRL methods. While the policy gradient methods are possible to implement using other function approximators, ANNs are commonly used due to their benefits.

Some other benefits of using DRL are generalizability, end-to-end learning and possibility for more complex policies. Generalizability stems from approximating functions with ANNs, which allows the agents to perform in unseen states and possibly even in completely different scenarios [3]. Again, using ANNs allows us to train the agents in an end-to-end fashion which potentially reduces the complexity of the implementation as we do not need steps such as feature selection. Lastly, with ANNs being able to represent non-linear functions and being able to work with high-dimensionality data, DRL methods are able to learn highly complex policies increasing the number of potential applications.

2.5 Hierarchical Reinforcement Learning

HRL extends the previously mentioned methods by introducing hierarchical structure to the policy. Rather than having a single policy taking the observation as input and outputting the action, we add policy levels where the output of the higher-level policies are used with the lower-level policies. In the simplest case, the output of a higher-level policy can be appended to some observations and fed into the lower-level policy as an input. With this in mind, HRL aims to learn simpler sub-behaviours in the lower-level policies that are composed into more complex behaviours with the use of the higher-level policies [3].

HRL uses two abstraction methods, temporal and state abstraction, to solve complex problems [3]. Temporal abstraction is concerned with constructing sub-behaviours from sequence of primitive actions, spanning multiple timesteps. This abstracts the low-level primitive actions, which on a larger scale may be inconsequential, into meaningful actions or sub-behaviours which simplifies the problem for the higher-level policies. The state abstraction is concerned on abstracting the state-space, which is what is generally done in DRL [3]. Combining these abstractions allows us to make decisions on the higher level which may involve multiple separate sub-behaviours. For example, a highest-level policy decides that to open a locked door, we need to first collect the key, unlock the door using the key and lastly open it. The policy next level down then is concerned on first selecting sub-behaviours to collect the key which may involve walking towards different directions and picking up the key, followed by selecting sub-behaviours to open the door.

This approach has multiple advantages compared to the other methods discussed

before, including credit assignment and exploration [3]. The credit assignment problem is concerned with what actions should be rewarded and how much, especially in environments with sparse rewards. Rather than assigning the individual primitive actions rewards, we assign rewards to the temporally abstracted sub-behaviours speeding the learning process of the value function [3]. Temporal abstraction can help with the exploration problem, as they can help explore the environment more widespread due to the temporally abstracted sub-behaviours being able to prevent the over-exploration of states near the starting state [3]. However, it also has been observed that with some sub-behaviours the exploration problem may get worse. For example, if all of the sub-behaviours only move back-and-forth, the agent may never explore anything but the immediate neighbourhood of the starting state.

There are multiple different ways to learn sub-behaviours and train the policies when to use them. Firstly, an expert can manually create the sub-behaviours which has the benefit of each of the sub-behaviours being meaningful and the effect of the sub-behaviour being known. However, in complex environments this can be prohibitively time consuming, nullifying a large benefit of using RL or HRL methods in the first place. Alternatively, the sub-behaviours can be learned automatically and this approach can be split into two main methods, staged or end-to-end. With the staged approach, either the sub-behaviours are first trained, followed by the higher-level policies learning to use the sub-behaviours in a bottom-up approach [3]. Alternatively, the higher-level policies are trained first followed by training the lower-level policies to discover sub-behaviours in a top-down approach. In the end-to-end approach the higher-level policies are trained simultaneously with the lower-level policies.

2.6 HRL Frameworks

There are three main frameworks for HRL methods, problem-specific models, options and goal-conditional.

Problem-specific models are highly specialized approaches of implementing HRL, and were historically used for demonstrating capabilities of HRL [3]. They often work by having several layers of sub-behaviours, for example in the Feudal Q-learning approach where the higher-level behaviours assign tasks for the lower-level behaviours until the lowest-level behaviour is given a task and then directly acts in the environment. Learning in problem-specific models is commonly achieved by selectively giving behaviours only certain parts of the observations and/or rewards such that the behaviour can only solve the task given by the higher-level behaviour. This approach has the benefit of the behaviours often being well defined and easily interpretable due to the significantly smaller task being solved. However, problem-specific models have the major drawback of difficulties in automatization of discovering appropriate selection of observations and/or rewards [3]. Due to these difficulties, problem-specific models commonly require expert knowledge on the task and manual design work, which can be highly time consuming and may even be infeasible in complex environments. Additionally, due to the problem-specific nature, the models are not easily transferable to other problems.

The options framework, on the other hand, requires much less manual design

compared to the problem-specific models and may simplify the structure compared to problem-specific models. It is based on having a list of options, that in essence are self-contained reinforcement learning policies or sub-behaviours, that can solve a specific part of the larger problem. For example, these options might be as simple as moving straight in one direction or a more complex combination of actions such as using a key on a door, opening it, and lastly walking through the open door. In addition to the options, there is a higher-level policy called policy-over-options that selects which option should be used depending on the observations, as can be seen in Figure 3. Commonly, once an option has been chosen, the option dictates the actions until a termination condition is reached [3]. Alternatively, other options could be tested for higher expected reward and subsequently, switched to if it is higher. A major design question with the options framework approach is the method for finding suitable termination conditions for the options. For example, manually placing goals for the options can help with the options being well defined, but requires expert knowledge and may be highly time consuming [3]. Some alternatives include states with high reward signal, frequently visited states on successful trajectories and states that unlock further progression, such as picking up a key. There are two main issues with the options framework, namely scalability with the number of options and inefficiencies in training options. Over a certain number of options, it is ever harder to increase the number of options, not to mention it compounding with the second issue [3]. As the options are essentially self-contained, they do not share any information even when it could be reused between the options, for example, the first few layers of a neural network. This causes significant issues with efficiency especially when the number of options or the size of the neural networks increase.

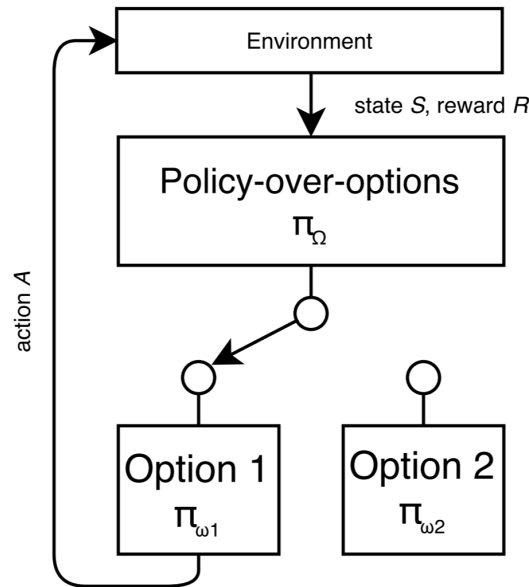


Figure 3: The options framework flowchart. Figure [3].

Lastly, goal-conditional framework simplifies the structure even further, replacing

the list of options with a single policy as can be seen in Figure 4. In this case, the policy-over-options is replaced with a higher-level policy that generates a goal-vector that is fed into the lower-level policy. The lower-level policy should then utilize the goal-vector to essentially select a sub-behaviour [3]. This time the major design decision is what the goal-vector should represent, either the full observations or some latent space [3]. Simply put, the lower-level policy attempts to reach the goal state such that the observations match the goal-vector and is rewarded accordingly. One of the main benefits compared to the options framework is the information sharing between sub-behaviours.

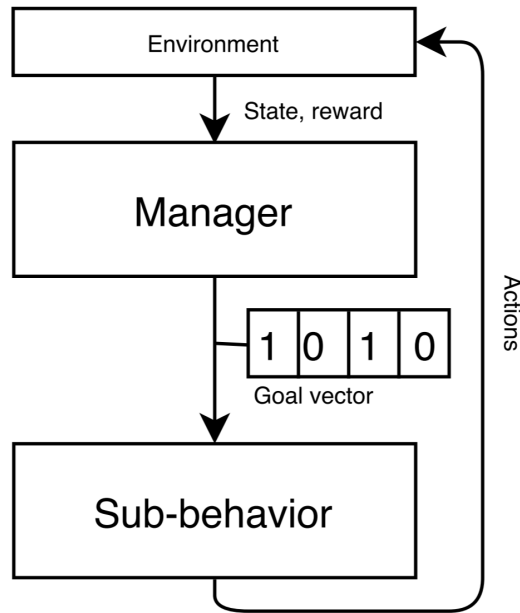


Figure 4: The goal-conditional framework flowchart. Figure [3].

2.7 Game Engines and RL environments

In order to train a RL agent, we need an environment for the agent to act in with sufficient complexity to produce interesting results. Some of the most common 3D physics simulators include Pybullet, MuJoCo and Unity [12]. Out of these, Unity is the most flexible due to the ability to utilize multiple different physics engines, such as the Bullet physics engine used by Pybullet or the MuJoCo physics engine [12]. Alongside this, the Unity editor includes tools for creating the environment in a visual way, whereas the environments for Pybullet [13] or MuJoCo [14] are created using XML files or inside the interface code.

In addition to the 3D physics simulation environments, there are multiple different environment tools that are more focused on 2D environments. However, our main interest lies with 3D environments due to the inherent simplicity of 2D environments.

To interface the RL code with the environment tool, we need a way to transfer observations, actions, and possibly rewards between them. To achieve this, we

generally use either of the two common application programming interface (API) standards, Gymnasium [15] and PettingZoo [16], which are created for single- and multi-agent RL respectively. These API standards provide easy and consistent way to interface with the environments and are generally usable with most environment tools. Some of the environment tools require additional libraries or pieces of software to be compatible with the API standards. Commonly, Unity ML-Agents [4] is used with the Unity game engine to provide support for the API, and likewise the Gymnasium-Robots [17] can be used to interface with the MuJoCo [14] physics simulator.

3 Methods

This section describes the methods and other design choices used in our experiment. We use the physics based locomotion and navigation environment described later in Section 5 as a case study to evaluate viability of using HRL in physics based video game environments. The choice of environment affects our choice of methods, as physics based locomotion requires significantly more complex behaviours compared to environments where the NPCs can be moved by simply changing their coordinates.

In Section 2.6 we described the three main HRL frameworks, including how they differ and some of their benefits and drawbacks. The problem-specific models were immediately disregarded due to their highly manual process, largely negating any of the benefits of using HRL. The choice between options and goal-conditional frameworks came largely down to two factors. First, using a single lower-level policy has a major benefit for our experiment, namely the information sharing, which could significantly increase sample efficiency and improve the training time required. Second, the options framework could face problems in more complex environments as they could require a significant number of options, which the options framework struggles with. As such, we chose to go forward with the goal-conditional framework.

In the first section, we first describe a goal-conditional HRL method we used as a basis for our own method. This is followed by a description of the modifications to the base method and some of the ideas behind the changes. Lastly, we describe other design decisions we made that are not strictly about HRL.

3.1 Base HRL Method

We chose an existing HRL method to base our method on to reduce the amount of design choices. Hierarchical reinforcement learning with off-policy correction (HIRO) is a goal-conditional hierarchical reinforcement learning framework and has shown promise in physics based locomotion problems [18]. The framework is based on a two-level hierarchy, consisting of a higher-level policy μ^{hi} and a lower-level policy μ^{lo} . The higher-level policy generates a goal-vector by sampling $g_t \sim \mu^{hi}(s_t)$ with the environment observations s_t every higher-level step, occurring every c training step. Between the higher-level steps the goal-vector is generated by a fixed goal transition function $g_t = h(s_{t-1}, g_{t-1}, s_t)$. The lower-level policy uses the environment observations s_t and the goal-vector g_t to sample atomic actions $a_t \sim \mu^{lo}(s_t, g_t)$ for the agent in the environment. The combination of current observation s_t and the goal-vector g_t results in a goal state that the lower-level policy attempts to match. These atomic actions a_t form temporally extended actions $a_{t:t+c-1}$ such that between two higher-level steps c there is one temporally extended action [19]. The environment agent uses the atomic actions which causes a state change in the environment resulting in a new state s_{t+1} and the environment returns a reward R_t [18].

The higher-level policy collects the rewards $R_{t:t+c-1}$ from the lower-level policy's steps as its own reward [18]. The lower-level policy is rewarded by the higher-level policy with an intrinsic reward produced by a fixed parameterized reward function

$r_t = r(s_t, g_t, a_t, s_{t+1})$. In this case the reward function is defined by

$$r(s_t, g_t, a_t, s_{t+1}) = -||s_t + g_t - s_{t+1}||_2,$$

which measures the distance between the new state and the goal state. This requires the goal-vector to change such that the goal state remains in the same position for every lower-level step. The change between states s_t and s_{t+1} is subtracted from the goal-vector and thus the fixed goal transition function is defined as

$$h(s_{t-1}, g_{t-1}, s_t) = s_t + g_t - s_{t+1}.$$

The transitions $(s_{t:t+c-1}, g_{t:t+c-1}, a_{t:t+c-1}, s_{t+c})$ and $(s_t, g_t, a_t, r_t, s_{t+1}, h(s_t, g_t, s_{t+1}))$ for the higher-level policy and lower-level policy respectively are stored for off-policy training [18]. However, due to the lower-level policy changing over time during the training process, there is a non-stationary problem with the higher-level policy. That is, during off-policy training the transitions conditioned on the same goals may have differences due to differences in the lower-level policies.

3.2 Our HRL Design

Our own design is a hierarchical two-level structure, with a higher-level policy and a lower-level policy. The design is inspired by and takes some attributes from goal-conditional and options hierarchical reinforcement learning frameworks, such as the HIRO [18] and Option-Critic [20] frameworks respectively. We use two-levels of hierarchy to have separate policies for making strategical decisions and for locomotion decisions. This approach for separating the strategy and locomotion has been shown to be effective and has benefits for both real-world and simulated scenarios [21]. Specifically for this thesis, the main benefit is learning a lower-level policy that generalizes well and can be transferred into other environments with a minimal effort. Additionally, as discussed in Section 2.5, decomposing the complex problem into longer term strategy and shorter term locomotion actions could make the learning process faster while allowing for the agent to learn more complex behaviours.

Our method uses information hiding, similarly to it being used in problem-specific and goal-conditional frameworks [3]. Information hiding is based on the idea of giving certain policies only a part of the observations. The overall goal with this approach is to guide the policy levels to focus on certain tasks, and additionally it may speed up the learning due to the reduced input size for the ANN. We discuss the specifics of the agent’s observations later in Section 5.1, but the overview can be seen in Figure 5 and are briefly mentioned below.

The higher-level policy is only concerned about deciding what overall behaviour the lower-level policy should follow, and as such we only give a part of the observations to the policy. As a part of the information hiding approach, we hide the observations regarding the specifics of the physical agent, as they are not relevant to the higher-level policy’s intended decision process. These hidden observations include information about the limbs’ angles and positions among other irrelevant data. The higher-level policy outputs a skill-vector for deciding on the skill that the lower-level policy should

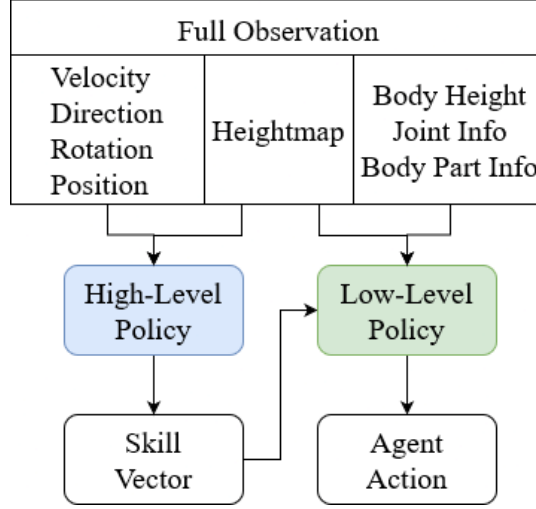


Figure 5: Our information hiding approach.

use, essentially working as a compromise between the goal-vector in goal-conditional and choice of option in options framework. The intention of this is to be able to distinguish between behaviours the lower-level policy has learned, essentially learning well-defined skills such as walking forward or jumping. This would make it possible to mask the skill-vector and would allow us to experiment with preventing certain skills from being used for creating agents with different levels of competency.

Conversely, the lower-level policy is only concerned about the agent’s locomotion without any regard to the long-term strategy. The lower-level policy is given the skill-vector produced by the higher-level policy alongside the observations about its body parts. The full observation is not given with the goal of preventing the lower-level policy simply disregarding the skill-vector and solving the problem on its own. We only give the lower-level policy observations that are directly beneficial for the locomotion task without giving information necessary for navigating to the goal. The lower-level policy has two termination conditions; the agent terminating inside the environment or the lower-level policy reaching a predefined maximum number of steps. In order to navigate successfully, the maximum number of lower-level steps can not be too high, because it would cause the agent to overshoot or take inappropriate actions for the current state. The lower-level policy outputs the atomic actions [18] for the agent to use at each physics timestep.

Lastly, the reward distribution is an important factor in having the agent learn meaningful behaviours. The higher-level policy collects the lower-level policy’s rewards $R_{t:t+c-1}$ from the high-level step c as its own reward as is done in HIRO [18]. This reward is used because the agent is mainly rewarded for moving towards the goal position, which requires the agent to use the correct skill to maximise the reward. This in turn causes the higher-level policy to learn to select the correct skill depending on the environmental observations regarding the obstacles, as the expected reward is higher for the correct skills compared to the other available skills. However, since we are not using the goal-vector as in HIRO [18], we can not use the same reward scheme

for the lower-level policy. Instead, we are also using the reward from the environment directly as the lower-level policy's reward due to the environment rewarding for taking actions that move the agent towards the goal.

4 Implementation

The goal of the implementation is to create a framework for our approach that can be utilized in a wide variety of environments. We aim to implement our method such that it can function in physics based 3D environments and supports masking the skill-vector to experiment with creating multiple different levels of competency.

The implementation of the project can be split up into two main parts, the Python HRL code and the Unity Environment. These parts can further be divided into individual components seen in the Figure 6. This section mainly focuses on the RL Python implementation side, whereas the Unity Environment side is mainly discussed in the Experiment Setup Section 5.1.

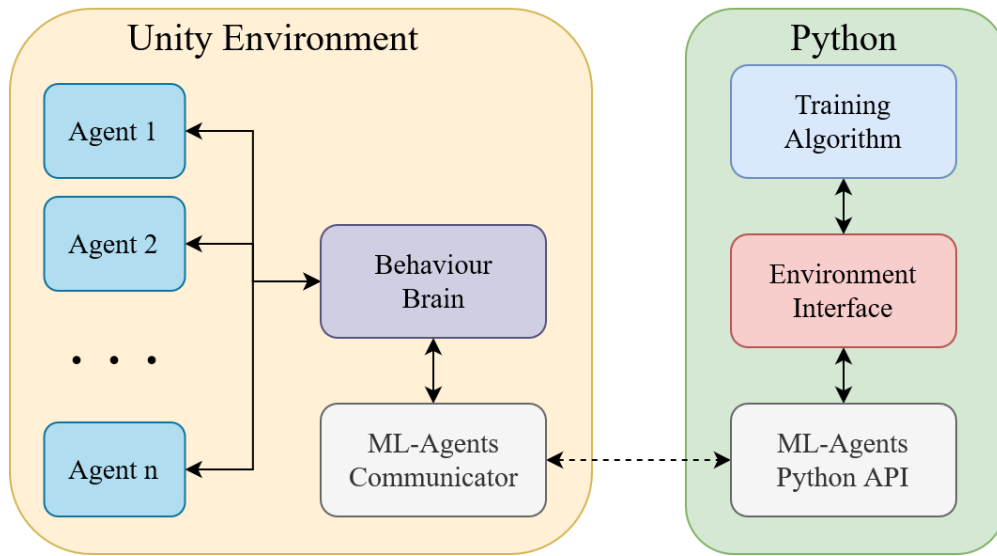


Figure 6: Components of the implementation visualized. The components with a light grey background are components which were contained either in the Unity ML-Agents package or the mlagents Python library and were not modified. The rest of the components were either created from scratch or modified from existing assets.

In this section we first focus on the HRL implementation and part of the interface code that is both largely agent and environment software agnostic. This is followed by a section describing the interface between the training code and the environment, in this case Ray RLlib and Unity ML-Agents. Lastly, we discuss some of the major issues encountered with Ray RLlib and how they were circumvented.

4.1 Implementation of our HRL method

We use Ray RLlib [5] reinforcement learning framework as the basis for the agent training implementation. The codebase can be mainly split in two parts, the training algorithm and the environment interface. First, we describe the main training implementation with an overview of the code and discuss some of the main decisions. This is followed by the structure and design of the step functions that make the HRL

implementation work. Lastly, we briefly describe the inference implementation used for collecting metrics from the trained models. The implementation discussed in this section can be made to work with other agents and environment software with little modifications to the code.

The training algorithm code mainly follows a common Ray RLlib implementation using the Ray Tune library. First, the environment interface is added to the Tune environment registry to be used later. This is followed by configuring the policies, including their observation and action spaces and optionally the neural network model to be used. A custom model from a machine learning framework may be specified here for greater flexibility rather than using a Ray RLlib generated model. The action space for the higher level policies here matches with the skill-vector discussed in the Section 3.2. As such, the action space from the higher level policy is appended to the lower level policy’s observation space. Additionally, a function is created for mapping the agent identifiers to the policy. These are followed by the normal Ray RLlib configuration options, including the machine learning framework, PPO hyperparameters and allocated resources. Lastly, we add a callback for logging the training process to the Tune object and call the function to start the training.

Some of the main decision points, besides those dependant on the specifics of the experiment, are the machine learning framework and the structure of the neural network. The choice of the machine learning framework should not make a large difference, but may impact the choice whether or not the chosen RL algorithm has support for the framework. However, in order to use graphical processing unit (GPU) acceleration with Tensorflow on Windows, we would either need to use an old Tensorflow version or move the training to Windows Subsystem for Linux (WSL). In addition, some NVIDIA software such as cuDNN and CUDA toolkit are required. As the development was partly done on Linux and partly on Windows, this additional complexity largely ruled out Tensorflow. Additionally, as we had the option to experiment with custom models which would be easier with more prior experience, we ended up choosing PyTorch. We use two fully connected feed forward neural networks, one each for the high- and low-level policies. The specific values used for the neural networks are discussed in Section 5.3.

To support two-level hierarchical policies implemented in the training algorithm, we create a new class extending the modified Unity3DEnv from Ray RLlib described in Section 4.2. The step function is replaced by three functions, a main step and two step functions for their respective policy levels. The interaction between the step functions for a single HRL agent is shown as a visualization in Figure 7. First, the policy level specific step functions are called inside the main step function with the full action dictionary and their results are collected. The step function results, observations, rewards, termination and truncated statuses and auxiliary information are joined. Lastly, checking whether all of the agents, including both high- and low-level, are either terminated or truncated and a flag is set if so to the respective status. The high-level step function loops through all of the high-level actions, in this case the action meaning a skill-vector, and saves it to be used later in the low-level step. Additionally, the the step counter of the low-level agent is reset to the maximum number of low-level steps. The observations are set for the low-level agent to contain

observations from the low-level agent’s last step joined with the new skill-vector and the rewards, termination and truncated statuses are reset. The low-level step function much like a normal step function, with a few modifications to make it work in the HRL setting. These modifications consist of setting the observations and collecting the rewards for the high-level agent when the low-level agent terminates.

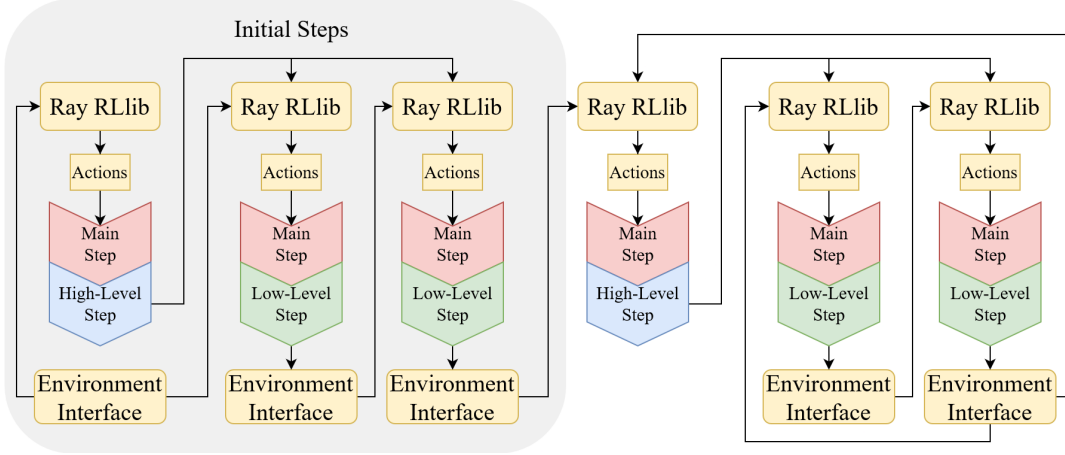


Figure 7: Flowchart visualizing the steps function in case where two low-level steps are taken for one high-level step. The left side with grey background represents the initial steps for each episode, with difference being the origin of observations used for the first high- and low-level step in each loop.

The inference code is largely the same as the training code, however, we do not use Tune in this instance. We encountered some issues with Tune during the evaluation process, hence we replicated much of the RL pipeline process by hand. We first initialize the environment, set the policies to be same as the trained one and load the checkpoint containing the policy models. We then reset the environment and start a loop that repeatedly computes the actions from the policy with the observations, calls the interface step function and resets the environment when all agents have terminated or a step limit is reached. The inference code does not collect metrics, but rather is used in conjunction with the environment and relies on the environment collecting the relevant metrics.

4.2 Interfacing Ray RLLib with Unity Game Engine

Ray RLLib includes an interface code, Unity3DEnv class, for connecting to Unity environments utilizing the API included in the mlagents Python library. The mlagents environment API is a low-level API for directly interacting with the Unity Environment, not suitable for using with most RL libraries as is. The interface code works as a wrapper for the low-level API and converts it to a multi-agent API suitable for Ray RLLib, similar to the common Gymnasium [15] API standard. We modify the Unity3DEnv class to include features and modify it to better suit our needs of the experiment and extend it to support HRL.

Due to the environment being a multi-agent environment, we need to consider that the agents act and terminate independently. This introduces an issue of multiple agents sharing the same episode, which will only reset if either all of the agents terminate simultaneously or the horizon is reached. Rather than having the agents wait for either of the conditions to be met after terminating, we would like to have all of the agents training for the whole duration of the episode. However, Ray RLlib can not handle a single agent having multiple different trajectories during a single episode. To circumvent this issue, we create an incrementing agent identifier system, where an additional digit is added to the identifier representing the times the agent has reset. This way, Ray RLlib considers the trajectories originating from multiple different agents. The alternative of having to wait for all of the agents to reset would greatly sacrifice the number of collected trajectories, especially the longer training goes on, slowing down the whole process.

Another addition we made was to use the sidechannels included in the low-level API, which allows us to interact and change configuration values inside the Unity Environment instance. The main focus of this was to increase the physics timescale, effectively allowing us to speed up the training process significantly without having to manually set it before compiling the environment. This allows us to keep the timescale at normal while developing iteratively to more easily observe the changes when running the environment in Unity Editor. Additionally, it allows us to control the capture frame rate and target frame rate to control the fixed update interval and to make the visualization refresh at higher frame rate respectively.

We extend the modified Unity3DEnv class for use with HRL, including the step functions described in Section 4.1. Due to the nature of needing to keep agent specific values, such as the skill-vector and number of low-level steps, between each step, we create dictionaries to hold the information. These dictionaries are then used inside the step functions and then reset inside the reset function. Similarly to the non-HRL case, we implement an incrementing identifier system to prevent multiple trajectories in a single episode. However, in this case we have to increment the identifier for the low-level agent much more often, as it is terminated after it reaches the maximum number of low-level steps.

4.3 Issues with Ray and RLlib

During the development process, we encountered numerous issues mainly related to Ray RLlib. A majority of the issues are not directly caused by the software itself, but rather the lackluster documentation. The encountered issues include debugging difficulties, GPU acceleration and overall difficulty using and finding information from the documentation. Some of the issues may not be relevant depending on the version of Ray RLlib, as there may be major differences in the documentation between versions.

Due to the distributed nature of Ray, and by extension RLlib, even when running on a local machine, the process is split into multiple worker nodes, essentially processes. In general, debugging multiprocess programs in Python is often difficult compared to a single process and/or thread debugging. Development environments such as Visual

Studio Code often include an option to enable debugging multiprocess programs. However, in case of Ray RLlib, the option has no effect and the breakpoints set in the code are ignored by the debugger. To solve this, Ray have created a debugging feature, however, it is not without its issues. Firstly, it requires adding function calls to the code itself to mark breakpoints rather than using the development environment breakpoints. Additionally, it does not work with the development environment debugger, but rather requires for the code to be debugged with an external commandline debugger. The reasons for these are most likely for enabling debugging on a distributed system, where a local debugger would not work. Lastly, the debugger is an experimental and unstable feature, which may cause issues of its own. A solution to debugging Ray RLlib on a local machine exist, however, the feature is deprecated and is not mentioned in the documentation for versions later than 1.13.0. The feature is a local mode option, which runs all code under a single process and as such the development environment debugger works. However, as mentioned, the feature is deprecated and is not confirmed to work in all cases and in later versions of Ray.

Another major issue encountered was with using GPU acceleration to speed up the training process. RLlib uses two main types of workers, normal workers and learning workers. The normal workers handle processing of the observations and sampling from the neural network among other things. The learner workers handle the model update loop, which in DRL may take significantly longer compared to sampling from the model. However, the documentation section discussing specifying and allocating resources in RLlib fails to mention the existence of the learner workers in both text and example code snippet. The documentation mentions setting the *num_gpus* value to use given amount of GPUs, but the default behaviour of assigning no GPUs to the learner workers and only assigning them to the normal workers is not mentioned. It may be even detrimental to use GPUs for the normal workers, as the observation data has to be transferred to the GPU's video random-access memory (VRAM), model sampled and then action transferred back to the central processing unit (CPU) random-access memory (RAM) to be used in Unity. The benefits of sampling the model on GPU may be outweighed by the additional time taken for transferring the data, and this was observed in our experiment. These issues were solved by allocating no GPUs to the normal workers and allocating them to the learner workers instead, resulting in approximately two to four times speedup in the training process.

As mentioned later in Section 5.3, we would have liked to use the Swish activation function in the neural networks. Using it worked when GPUs were not assigned to the learner workers, but once GPUs were enabled the training process kept crashing sooner or later due to arrays being filled with NaN values. This seemed to occur due to the exploding gradients problem, and as such there seems to be an issue with either the GPU based learner workers or the Swish activation function implementation. It could also be related to PyTorch and could possibly be fixed by using Tensorflow instead. Due to these issues, we end up using the default Tanh activation function. Similar to the activation function, the learning rate scheduler fails to work when GPUs are enabled on the learner workers with PyTorch framework.

The organization of the documentation is also problematic in some cases. For example, the RL algorithms and other configurable options have a large list of possible

parameters that can be given on top of the inherited parameters from the parent class. Due to this, however, the common parameters from the parent classes are not listed but rather a link to the common parameters is given. The link to the common parameters leads to the getting started page with a section tag in the URL to automatically scroll down to the relevant section. However, the section tag has not matched for several versions resulting in the page not scrolling down, causing unnecessary friction navigating to the relevant section. Additionally, it is unclear whether getting started page is the right place for a massive list of parameters spanning across all types of configuration options in RLlib instead of it having its own page.

Lastly, there is a major issue on finding the default values for the parameters listed. In many cases, there first is a list with all of the parameters with their datatype and default value. This is followed by another list containing the names of the parameters with description of the parameter, with examples in some cases. However, in majority of the lists, the default value is marked as a *NotProvided object* as can be seen in Figure 8. This likely is caused by the documentation being generated automatically from the source code, where the *NotProvided objects* are listed as default values in the function. The documentation has a link to the appropriate section of the source code where in some cases a selection of the values for the parameters can be found, but it is not certain whether the values get overridden at some later point in the code. Due to these issues, oftentimes it is easier to find the default values by running RLlib in the local mode with debugging in a development environment. This way, the program can be paused before the parameters are assigned any user-defined values and the default values can be determined. However, due to the large number of parameters this method is not particularly convenient either, with PPO configuration having total of 153 parameters.

```

training(*, lr_schedule: ~typing.List[~typing.List[int | float]] | None =
<ray.rllib.utils.from_config._NotProvided object>, use_critic: bool | None =
<ray.rllib.utils.from_config._NotProvided object>, use_gae: bool | None =
<ray.rllib.utils.from_config._NotProvided object>, lambda_: float | None =
<ray.rllib.utils.from_config._NotProvided object>, use_kl_loss: bool | None =
<ray.rllib.utils.from_config._NotProvided object>, kl_coeff: float | None =
<ray.rllib.utils.from_config._NotProvided object>, kl_target: float | None =
<ray.rllib.utils.from_config._NotProvided object>, sgd_minibatch_size: int |
None = <ray.rllib.utils.from_config._NotProvided object>, num_sgd_iter: int |
None = <ray.rllib.utils.from_config._NotProvided object>, shuffle_sequences:
bool | None = <ray.rllib.utils.from_config._NotProvided object>,
vf_loss_coeff: float | None = <ray.rllib.utils.from_config._NotProvided
object>, entropy_coeff: float | None =
<ray.rllib.utils.from_config._NotProvided object>, entropy_coeff_schedule:
~typing.List[~typing.List[int | float]] | None =
<ray.rllib.utils.from_config._NotProvided object>, clip_param: float | None =
<ray.rllib.utils.from_config._NotProvided object>, vf_clip_param: float | None
= <ray.rllib.utils.from_config._NotProvided object>, grad_clip: float | None =
<ray.rllib.utils.from_config._NotProvided object>, vf_share_layers=-1,
**kwargs) → PPOConfig \[source\]

```

Sets the training related configuration.

Figure 8: Part of parameter listing for PPO with not provided default values in Ray RLLib documentation.

5 Experiment Setup

The primary goal of the experiment is to assess the viability of using HRL for creating non-player character (NPC) behaviour in video games. Specifically, the experiment aims to evaluate the performance of a physics based locomotion and navigation tasks in controlled scenarios that can be expanded into more complex scenarios. Our HRL method described in Section 3.2 is evaluated against more traditional RL methods from Ray RLlib and ML-Agents frameworks. Before this, we compare the traditional RL methods against each other.

We first train and evaluate the PPO implementation in the Unity ML-Agents [4] as a baseline to verify and compare our results against. This is followed by PPO implementation in Ray RLlib [4] to verify our environment interface and compare the differences between ML-Agents and RLlib in traditional RL. We then train and evaluate an agent using the HRL implementation discussed in Section 4.1 where all of the observations are given to both of the policy levels. Lastly, we repeat the HRL experiment, but we split the observations into two categories and limit what the higher and lower level policies are given.

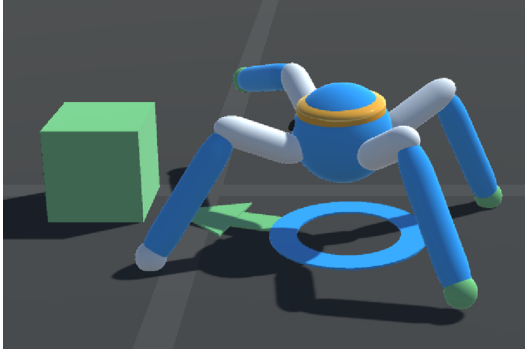
The two chosen HRL approaches are done to evaluate the impact of manually engineering the observations for a given policy level as opposed to having the policies learn on their own. Additionally, it promotes the lower-level policy using the skill-vector generated by the higher-level policy rather than just ignoring it and essentially reverting to traditional RL. Lastly, we test the implementations in a more complex environment with obstacles.

In this section we first describe the environment in detail focusing on both the environment itself and the agent, including its observations. This is followed by description of the metrics used to evaluate the performance of the agents. Lastly, we discuss the hyperparameters and configurations used in the training process.

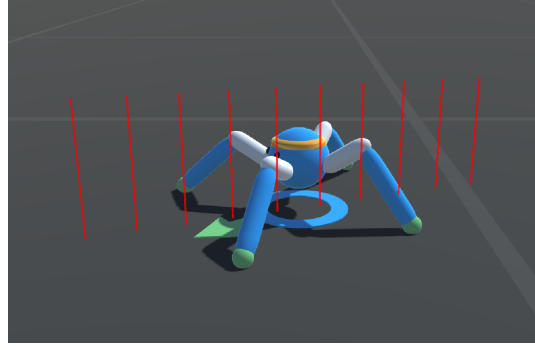
5.1 Environment Description and Implementation

In order to experiment with HRL-based agents, we need to define an environment to use as an case study for physics based agents. The environment is based on the Unity ML-Agents [4] Crawler environment with modifications made to suit the experiments. We use a modified version of the crawler agent, shown in Figure 9a, with custom environments. The crawler agent consists of a singular body and four legs, with each leg having a hip and a knee joint. The hip joints can rotate in both yaw and pitch axis, i.e. sideways and up and down respectively, while the knee joint can only rotate in the pitch axis. This environment is a locomotion problem with additional navigation requirements to reach the target.

The observations remain largely unchanged from the default observations in the crawler example environment. The crawler has 51+126 observations, where the first set represent more general observations and the latter set represent individual body part observations. These first observations include average body velocity, target direction, rotation delta between the agent and the target, the agent’s relative position to the target, distance from body to ground, ground contacts, joint strengths and raycasts.



(a) Crawler agent from Unity ML-Agents [4] with a target object.



(b) Crawler agent with visualizations depicting the raycast observations pointing downwards.

The body part observations include velocity, angular velocity, rotation and position of the individual body parts. For additional information about the environment, we add a one dimensional grid of raycasts pointing both up and down. These raycasts are meant to be used for identifying the object in front of the agent and also help the agent to position the legs, for example, where to place a foot on a stair. A visualization of the downward pointing raycast observations can be seen in Figure 9b.

We implement two different approaches for feeding the observations to the different policy levels. In the first approach the full observations are given to both of the neural networks regardless of the policy level. The observations may contain information that is not useful for the given policy. In the second approach we manually select what observations are given to which policy level as discussed in Section 3.2 and visualized in the Figure 5. As mentioned, the navigational data is given to the higher-level policy, whereas data relevant to locomotion such as the joint information are given to the lower-level policy. The goal of this approach is to force the high-level policy to generate skills based on navigating to the target and the low-level policy to focus on locomotion.

The agent is given extrinsic reward signals, in this case rewarding the agent depending on its speed towards the target. The original crawler is set to have a randomized target speed that the agent attempts to match for creating an agent that performs well at multiple different requested speeds. However, as we want to compare the agent performance using the metric of time to reach the target, we set the target speed at a constant value slightly above what the agent can realistically achieve. The goal of this is to simplify the training process and to increase agent performance at the maximum speed. The reward for matching the target speed is multiplied by a value between 1 and 0, controlled by the relative angle between the direction the agent is facing and the target. This is done to promote the agent walking naturally, facing towards the target, which is necessary for the raycast detectors to function as intended. We experimented with alternative reward schemes, however, it often resulted in the agent learning to game the reward or other unwanted behaviour. The main goal of these alternative rewards was to allow the agent to perform better in other scenarios where there may be obstacles between the agent and the target. These reward schemes included distance delta between timesteps and conditional rewards depending on

whether the agent is moving towards the target among other smaller tweaks to the existing velocity based reward. However, the alternative reward schemes either did not increase the performance or added loopholes into the reward structure that the agent learned to game and exploit.

In addition to the flat environment with a dynamic target, we created an environment with multiple different types of obstacles to challenge the agent in ways not possible within the flat environment. In this more challenging environment, the obstacles introduce higher level of complexity to the navigation and decision making skills. The obstacles include gaps in the floor, stairs, large blocks and low ceilings, as shown in Figure 10. These obstacles require skills to navigate past them, jumping horizontally, climbing stairs, jumping vertically and crouching respectively. The environment is split up in multiple sections with a single agent in each section. These sections include one of the obstacles, such that the agent can learn to cross the obstacle without having to consider the navigational aspect as much and can focus on the decision making on what skill to use. Additionally, a section without any obstacles is included with the goal of promoting the agent to learn to walk normally. This should increase the training performance as the agents should learn to walk up to the obstacles faster, after which they can focus on learning to cross the obstacles.

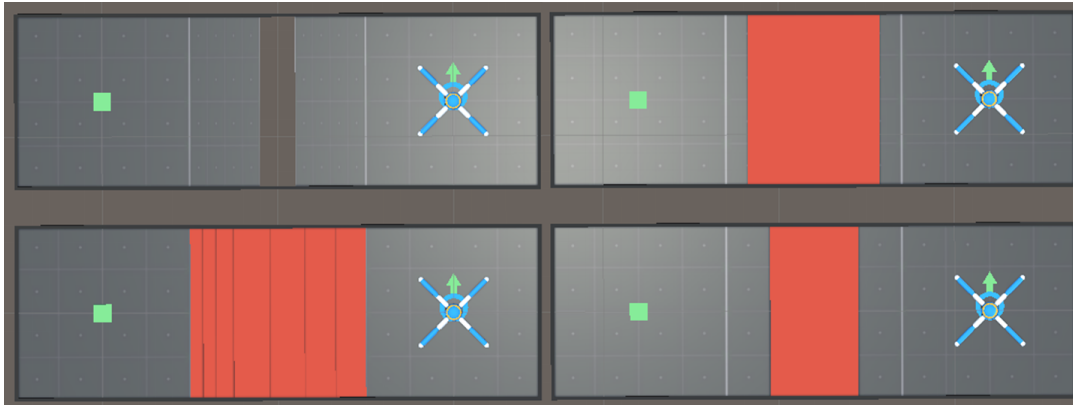


Figure 10: Obstacle environment including a gap (top left), stairs (bottom left), low ceiling (top right) and large block (bottom right).

5.2 Experiment Metrics

We collect metrics in two phases; during the training process and after the training with inference to collect additional metrics. The training produces mean episode rewards each training step, which can be used to compare between the different Ray RLLib implementations. However, due to differences in how the episode rewards are collected, it is not directly comparable to the ML-Agents implementation. As such, to compare between the different frameworks, we measure metrics inside the Unity game engine to provide us comparable metrics. We measure metrics such as speed towards the target, number of steps before termination and number of targets collected in a given number of timesteps inside the environment to produce comparable metrics.

This is done in an environment with a constant random seed such that the target spawns in the same location run-to-run.

However, as discussed later in Section 6.3, we were unable to properly run our HRL agents with inference. Due to this, we have to compare the HRL agents and the Ray RLlib baseline agent using the mean episode rewards collected during training. While this is not as accurate as metrics collected during inference, they still provide insight into the agent’s performance.

5.3 Experiment Details, Hyperparameters and Configuration Options

We use the Proximal Policy Optimization (PPO) [8] as the underlying training algorithm. PPO was selected for its simplicity in terms of implementation and computation, hyperparameter stability and good performance in similar environments, such as the ML-Agents [4] Crawler environment. Another option would have been to use TD3 [9] off-policy method, as is used in HIRO [18] to achieve high sample-efficiency as the HIRO framework is designed to be applicable in real-world scenarios where sample-efficiency is a significant issue. However, in our scenario with the environment being a simulation, sample-efficiency is not as important factor due to the ability to trivially generate large quantities of samples. As discussed in the background Section 2.3, off-policy methods can use stored experiences, however they may not be relevant to the current state of training. To address this, HIRO [18] introduces an off-policy correction algorithm for correcting the trajectories originating from old policies. However, the off-policy correction approach introduces significant implementation complexity, which we avoid due to using an on-policy method.

We use the ML-Agents [4] Crawler PPO configuration as a starting point for the hyperparameters and change them to suit our needs, accounting for the differences between ML-Agents and Ray RLlib. Mainly, ML-Agents uses the PPO-Clip variant whereas RLlib implements a combination of PPO-Clip and PPO-Penalty mentioned in the PPO article [8]. As such, the combined implementation uses both a clipping parameter and KL divergence parameters. The PPO hyperparameters used for the experiments can be found in Table 1. These hyperparameters were tuned using a process of trial-and-error. Optimally, we would have used a learning rate scheduler to maximize the speed of learning at the beginning of the training and have the learning rate decay as the training progresses to stabilize the learning. This could be especially useful with HRL as often the whole training process is more complex and/or overall, there are more parameters to optimize. However, as discussed in Section 4.3, we are not able to use learning rate scheduler and use a constant learning rate and compromise between learning performance and stability.

We use fully-connected feed-forward neural networks for both of the policies to match the implementations of HIRO [18] and ML-Agents [4]. The hyperparameters for the lower-level policy’s neural network are matched to the ML-Agents [4] Crawler example environment’s hyperparameters, with number of hidden layers set at three and number of units in each hidden layer at 512 as seen in Figure 2. Due to the relatively

small amount of added inputs to the lower-level policy’s neural network compared to the baseline methods, we forgo increasing the neural network’s size for consistency. As discussed in Section 4.3, we had issues with using the Swish activation function that ML-Agents [4] uses. Thus, we use the Tanh activation function in both of the neural networks, as it has been shown to function well in smaller neural networks and is the default option in multiple frameworks, including HIRO [18]. Other relevant hyperparameters, such as the allocated resources, can also be found in Table 2.

Hyperparameter	Value
Horizon	1000
Timesteps ¹	4 000 000
Learning rate	0.0001
GAE parameter (λ)	0.95
Discount (γ)	0.995
Entropy coefficient ²	[0.0002, 0.0]
SGD minibatch	1000 * 4
SGD iterations	3
Training buffer	1000 * 4
Clipping parameter	0.3
KL target	0.01
KL coefficient	0.2

Table 1: PPO hyperparameters used for the training processes in our implementation.

¹Timesteps are measured by environment steps, multiply by number of agents per instance to get agent steps. ²Entropy coefficient decays linearly during the training, reaching 0 at 20 000 000 agent steps.

Hyperparameter	Value
Environment instances	4
Agents per instance	10
Workers	4
Learner workers ¹	0
GPUs	1
GPUs per learner worker	1
Rollout length	100
Neural network hidden layers	3
Hidden layer sizes	512 nodes

Table 2: Other relevant hyperparameters and configuration options used in the experiment. ¹Setting number of learner workers to 0 uses the head process for learning.

The main software and library versions used in the experiment are listed in Table 3. The computer specifications used for training all of the models are listed in Table 4.

Software/Library	Version
Python	3.9
Ray RLlib	2.8
mlagents-envs	0.30.0
mlagents	0.30.0
PyTorch + CUDA	2.10 + 12.1

Table 3: The main software and python library versions used in the experiment. Miscellaneous libraries not listed.

Component	Model
CPU	Intel Core i7-10700K (8 core, 16 thread)
RAM	32GB (3200MHz DDR4)
GPU	NVIDIA GeForce RTX 3070 Ti (8GB VRAM)

Table 4: Relevant computer specifications used to train the agents.

6 Results

In this section we show and discuss the results of the experiments. We first take a look at the comparison between ML-Agents and Ray RLlib baseline agents. This is followed up by discussion about the results of the two HRL experiments. Lastly, we discuss findings about Ray RLlib not related to the agent performance.

6.1 Baseline without HRL

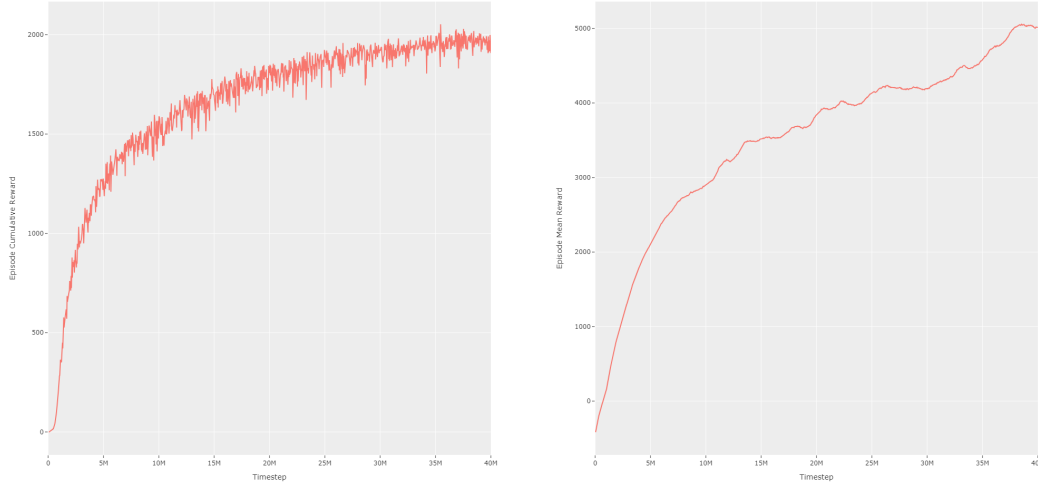
We first compare the results from the simple cases without HRL, the Unity ML-Agents default PPO Crawler training configuration and our baseline Ray RLlib PPO implementation. The graphs are not directly comparable, as the training progress is measured differently in Unity ML-Agents and Ray RLlib. However, the shapes of the graphs in Figures 11a and 11b for ML-Agents and RLlib respectively can offer an insight into the training performance. Notably, ML-Agents seems to reach approximately 90% of the final training performance at the halfway point, whereas RLlib reaches approximately 75% of the final training performance at the halfway point. Based on these training statistics and post-training evaluation discussed later, ML-Agents seems to have a lower sample-throughput while having higher sample efficiency resulting in a higher quality model. As discussed in Sections 5.3 and 4.3, we are unable to use learning rate scheduler with RLlib and use a constant learning rate instead. This resulted us in having to compromise with having the agent learn in a reasonable time, while still having the training be somewhat stable, especially at the end of the training. This may be the major contributor to the somewhat significant difference between the training performances between the two implementations. However, it is also notable that ML-Agents took slightly longer in realtime to reach the maximum amount of timesteps (Table 1) finish training compared to RLlib, at 357 minutes and 299 minutes respectively.

Metric	(Mean $\pm$ SD)	
	ML-Agents	RLlib
Steps ¹	4908.3 $\pm$ 557.2	4501.4 $\pm$ 1229.1
Speed	0.092 $\pm$ 0.020	0.044 $\pm$ 0.027
Collected	14.6 $\pm$ 2.2	6.3 $\pm$ 2.0

Table 5: Metrics from ML-Agents and RLlib baselines, averaged from 500 episodes.

¹The maximum number of steps in one evaluation episode is 5000.

In addition to the training progress graphs, we can compare the metrics obtained with the trained models to compare the performance of the agents. As can be seen in Table 5, there are significant differences in the agent performance. While the average number of steps in the evaluation episodes are somewhat similar, the standard deviation for the RLlib baseline model is significantly higher. This hints at the RLlib trained model being significantly more inconsistent and terminating earlier more often. In addition to this, the ML-Agents trained model is approximately 2.1 times faster



(a) Unity ML-Agents: Cumulative episode reward from default PPO model. **(b) Ray RLlib:** Mean episode reward from PPO baseline model.

Figure 11: Training performance graphs for the two baseline implementations. Timesteps measured in number of agent steps.

and collects approximately 2.3 times more targets on average. It was also observed that in some of the episodes the average speed was negative for the model trained using RLlib. This hints that the agent started moving away from the target either after collecting one or no targets and terminated shortly after.

There are differences between the PPO implementations in ML-Agents and RLlib, making it impossible to fully match the hyperparameters used in the training. This may contribute to the difference in the results, but we do not expect it to have such a large effect as observed. A more likely contributor to the worse performance of RLlib trained model is the constant learning rate mentioned above, notably it being too large to properly stabilise. Too large learning rate may exhibit itself as the observed erratic behaviour, where the agent flips on its head for no apparent reason or the jittery limb movement. On the other hand, a too small learning rate would significantly slow down the training unnecessarily and risks the training getting stuck in a local minima. Lastly, the inference methods differ between ML-Agents and RLlib, as ML-Agents produces a model file usable within the Unity environment, running directly inside the game engine. RLlib differs by the model being incompatible with the Unity environment without manually modifying the produced model file, but rather the inference code is used to sample the actions and use the same interface code as during the training. This issue and findings are further discussed in Section 6.3.

6.2 Our HRL Approach Findings

We compare the HRL results from two main perspectives, comparing the two different HRL approaches to the baseline RLlib results and between each other. In addition to this, we briefly discuss the more complex environment with obstacles.

Comparison between the RLlib baseline and the HRL implementation with full observations in both policy levels can be seen in Figure 12. We can see from the figures that full observation HRL implementation had slightly higher episode mean reward at the end of the training than the baseline model, at approximately 6363 and 5014 respectively. This difference is approximately 27% increase in the episode mean reward. Assuming that the performance of the agent increases linearly with the increase in the reward, we could expect the mean speed to be close to 0.056 and the mean collected targets be close to 8. The training time for this implementation was 421 minutes compared to the baseline's 299 minutes.

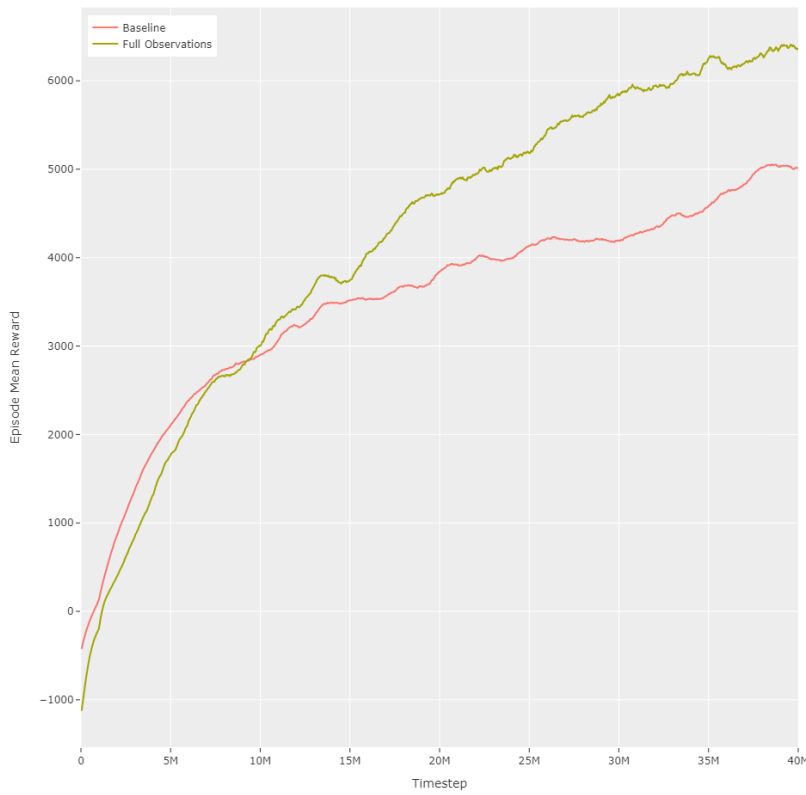


Figure 12: Training performance graph for the baseline and full observation HRL implementations. Timesteps measured in number of agent steps.

Comparison between the RLlib baseline and the HRL implementation with limited observations can be seen in Figure 13. Again, we can see a slight increase in episode mean reward, however at slightly lower for the limited observations compared to the full observations, with it being approximately 5930. The difference between the two approaches are close enough that we can assume the difference to be error due to randomness introduced in the training process, and running the training process would yield slightly different results again. As for the difference between the baseline, we see approximately 18% increase in episode mean reward. The training time for this implementation was 420 minutes compared to the baseline's 299 minutes.

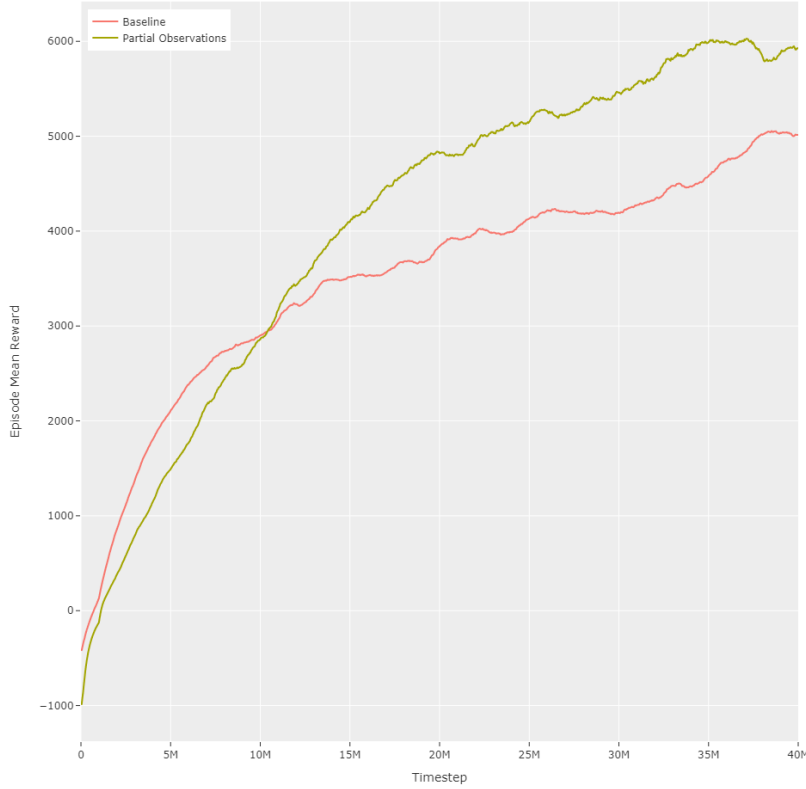


Figure 13: Training performance graph for the baseline and limited observation HRL implementations. Timesteps measured in number of agent steps.

Extracting meaningful results from either of the HRL implementations proved more difficult, as they exhibited greatly different behaviour during inference to what was observed during the training process. The behaviour during inference was erratic with the agents moving the limbs seemingly randomly, often resulting in the agent’s main body touching the ground and resetting. This would need further engineering to fix, either significantly modifying the Unity model runtime or RLlib inference code and model pipeline. Due to this issue, we are not able to collect and compare post-training metrics for the HRL approaches as was done for the baseline methods. Consequently the lack of consistent inference for the HRL approaches makes experimenting with the skill-vector masking unfeasible.

All three models perform adequately in the flat environment, with it being notable that none of the models’ results were close to the agent trained using ML-Agents. Additionally, with the HRL approach results not being evaluated with the metrics, we need to be critical of the training performance results, as they may not be fully representative of the actual performance of the agents.

Considering how close the results between the Ray RLlib baseline and the HRL implementations are, we argue that the environment is too simple to see the true benefits of HRL and the information hiding approach. However, using the more

complex obstacle environment, we observed that none of the approaches, including ML-Agents, made proper progress during the training. The agents made some progress at the very start, however once they started reaching the obstacles, the training would grind to a halt. This is most likely due to the environment being too difficult to initially learn anything besides walking and could benefit from alternative training methods, such as curriculum learning.

6.3 Ray RLlib Observations

As mentioned in Sections 6.1 and 6.2, there are issues with running the models for inference. Due to the complex nature of Ray, Ray Tune and Ray RLlib, the exact root causes of the issues are difficult to find. These include processes such as observation preprocessing or action postprocessing after sampling. As such, it was observed that the actions and the skill-vector had values outside of the set limits, which will most likely have adverse effects on the behaviour of the agent. Workarounds for this were investigated, including using the Ray Tune originally used for training to run the inference by resuming the training and forcing the learning rate to be zero. The Ray Tune has a function to restore previous training processes, however, it is not possible to restore a finished training process. Instead, Ray Tune documentation has some instructions on loading checkpoints into a new Tune experiment, which did not work as documented.

We additionally measured the effect of GPU acceleration on the training process. As discussed in Section 4.3, there are issues especially within the documentation detailing the usage of GPU acceleration for the training process. This may result in the agents being trained on the CPU rather than the GPU which has a significant impact on the training performance. As can be seen in Figure 14, there is a large difference between the graphs indicating improved training performance by using GPU acceleration on the learner worker rather than the normal worker process. The observed training performance difference is between two to four times reduction in time taken to train for the same number of steps. It is also notable that when using GPU with the normal workers, the iteration time starts out much higher but levels out at approximately twice as slow compared to using GPU on the learner workers.

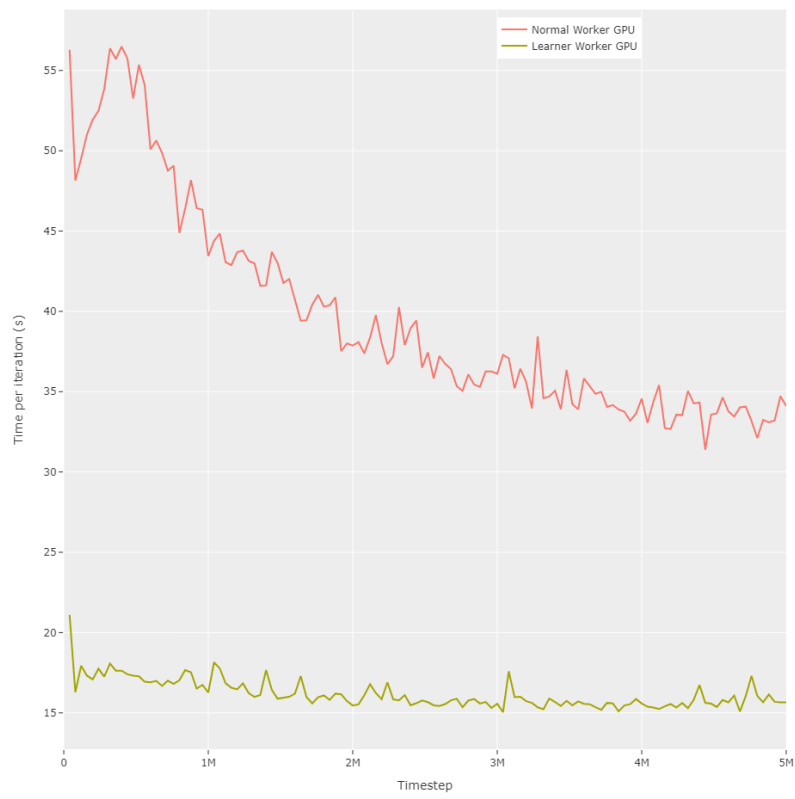


Figure 14: Iteration time comparison between using GPU acceleration on a learner and a normal worker. Timesteps measured in number of agent steps.

7 Discussion and Future Work

Video games are a massive industry with significant growth potential [22]. The industry includes the games themselves alongside hardware such as video game consoles and a broad range of accessories. The market revenue was 265.21 billion U.S. dollars in 2019, growing to 406.2 billion dollars in 2023 and has been forecast to grow up to 666.69 billion dollars by 2029 [22].

Traditionally NPCs in video games are created using hard coded behaviours, which often may require a significant effort and time from an expert developer [2]. Furthermore, creating NPCs with multiple different levels of competencies increases the workload as only parts of the previously created NPCs may be able to be used. If the approach used in this thesis were applicable to real-world video game development scenarios, it would significantly reduce the time and effort put into creating NPCs. While the benefits could be massive, it is not a silver bullet that would work for all situations. It requires effort of its own to make it feasible as discussed later in the future work Section 7.1.

Our approach is similar to other HRL methods, however, it in theory also offers an unique benefit of being able to create NPCs multiple levels of competencies with significantly less time required to train new ones. Other HRL methods could effectively require a NPC to be fully retrained and could additionally require alternative reward signals to promote different enough behaviours. This would demand significantly higher effort compared to our approach of masking the skill-vector and retraining the higher-level policy to achieve similar result. Our approach also partially addresses the problem of lifelong learning, where learned skills are reused and learning new skills is based on old skills [23] [3]. Provided that the new environment is similar enough and uses the similar lower-level skills, simply retraining the higher-level to use the skills in a different order or sequence may be enough to solve the problem. For example, the behaviours that other approaches, such as HIRO [18], can learn may be too complex to be reused in other environments, effectively preventing the behaviours from being used. As opposed to this, our approach intends to learn small enough and distinct skills such that they can be reused in new environments in different sequences.

In addition to the issues with our HRL approach, the problems with Ray RLlib seem to significantly contribute to the agent performance issues. The most significant problem most likely is the inability to use learning rate scheduler with GPU Acceleration. This prevents the training from quickly learning at the start of the process and then slowing down training at the end to stabilize the behaviour. The issues call into question whether Ray RLlib is the best option for implementing HRL methods. As it turns out, our implementation for HRL mainly focuses on the interface code, as it handles all the policy-level steps. It seems that the code that directly interacts with Ray RLlib could be rewritten to work with other RL training frameworks with not that much work.

On a positive note, our HRL interface code seems to work well and could support other HRL methods, such as HIRO [18], with minimal implementation effort. We think that with some future work, an interface code similar to our implementation could be standardized for HRL, similarly to the Gymnasium [15] or the PettingZoo[16] APIs. As it stands, there seems to be a significant lack of support for HRL in existing

RL software frameworks. A standardized API for HRL could lessen the effort required for researching and implementing new HRL methods.

It seems that the approach used in this thesis has not been used before in the literature that was reviewed. While the results show potential, they are not significant enough on their own for the approach to be used as is and requires further development.

7.1 Future Work

As discussed in the results Section 6.2 the environment with obstacles proved too difficult for our method to handle as is. The main approach we would take to solve this would be to include curriculum learning in the training process. This way, the agent could be trained in significantly easier environments such that the agent makes progress. Once the agent handles the easier environment, an ever more difficult environment could be used until the agent is finally able to behave appropriately in the hardest environment. This ties into another point of future work discussed next.

We could create an extensive environment that can be modified dynamically and would encompass a large suite of possible physics based tasks. The environment could contain obstacles or tasks which difficulty could be adjusted depending on the training progress, for example, making the steps in a staircase lower so that the agent can more easily climb the stairs. In addition to training, this could be useful for evaluating how well trained agents generalize into new tasks or how well new skills could be learned by an already trained agent while still retaining the previously learned skills.

Another improvement to the training process could be to implement an automatic tuning for the hyperparameters. It could simply be a grid search, or a more complex system such as PBT, briefly discussed in the background Section 2.3. This would most likely find more optimal hyperparameters, especially with it learning curriculum for the hyperparameters rather than them being static. Curriculum learning would most likely be best paired with PBT, as the optimal hyperparameters may change during the training process, especially with a changing environment.

Lastly, we would explore alternative HRL training approaches rather than training both policy levels simultaneously. Training the lower-level policy first with pre-defined skill-vectors could be used to obtain better defined skills, which would be beneficial for creating multiple levels of competency. This has the drawback of possibly training skills that are not needed, hence wasting training time [3]. Alternatively, training the higher-level policy first could be used to teach the higher-level policy to generate a different skill-vector for different observations. However, it is uncertain what it exactly would entail due to our approach of using a skill-vector rather than a goal-vector.

8 Conclusions

The goal of the thesis was to assess the viability of using HRL methods to create NPCs in video games. While traditional RL methods have shown promise, they often fall short in more complex video games [2]. HRL has an answer to some of the major issues and has potential for other major benefits, including improved training times and allowing for creation of multiple skill competency levels. These skill competency levels would even further reduce the time required to train multiple NPCs for different difficulties in video games.

In this thesis the author has designed, implemented, and studied a HRL approach that attempts to compromise between goal-conditional and options frameworks. To use HRL methods with Ray RLlib, the author modified the interface code between Ray RLlib and the Unity game engine, such that it supports multiple levels of policies. The design choices were made such that the method would allow us to gain some of the benefits of goal-conditional framework stemming from the shared lower-level policy. However, the goal-vector was repurposed as a skill-vector, which could allow us to mask some of the skills, resulting in multiple skill competency levels.

The results Section 6 shows potential that using HRL can provide benefits even with our simple approach compared to more traditional RL. However, this comes with the caveat that the methods may require more complex training methods to fully gain the benefits of HRL. The caveat is especially true in more difficult and complex environments, which require more sophisticated behaviours to reach the goal. As it stands, the effectiveness of our HRL method comes into question and it is not certain whether some of the design choices we made are viable. We discussed some potential improvements and changes to the approach in the future work Section 7.1.

Lastly, we discovered several significant issues with implementing HRL with Ray RLlib, ranging from documentation to implementation difficulties. These issues and the complexity of Ray RLlib lead us to believe that it may not be optimal out-of-the-box and could require significantly more effort compared to other RL frameworks. Furthermore, we have shown that Ray RLlib seems to produce significantly worse performing agents compared to ones trained using ML-Agents in our environment when using similar hyperparameters.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. The MIT Press, 2018. [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>
- [2] I. Millington, *AI for Games, Third Edition*. CRC Press, 2019. [Online]. Available: <https://doi.org/10.1201/9781351053303>
- [3] M. Hutsebaut-Buysse, K. Mets, and S. Latré, “Hierarchical reinforcement learning: A survey and open research challenges,” *Machine Learning and Knowledge Extraction* 2022, Vol. 4, Pages 172-221, vol. 4, pp. 172–221, 2 2022. [Online]. Available: <https://www.mdpi.com/2504-4990/4/1/9>
- [4] A. Juliani, V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Henry, M. Mattar, and D. Lange, “Unity: A general platform for intelligent agents,” *arXiv preprint arXiv:1809.02627*, 2020. [Online]. Available: <https://arxiv.org/pdf/1809.02627.pdf>
- [5] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. E. Gonzalez, M. I. Jordan, and I. Stoica, “RLlib: Abstractions for distributed reinforcement learning,” in *International Conference on Machine Learning (ICML)*, 2018.
- [6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [7] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, and Q. Miao, “Deep reinforcement learning: A survey,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 5064–5078, 2024. [Online]. Available: <https://doi.org/10.1109/TNNLS.2022.3207346>
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [9] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” 2018. [Online]. Available: <https://arxiv.org/abs/1802.09477>
- [10] M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals, T. Green, I. Dunning, K. Simonyan, C. Fernando, and K. Kavukcuoglu, “Population based training of neural networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1711.09846>
- [11] M. Kiran and M. Ozyildirim, “Hyperparameter tuning for deep reinforcement learning applications,” 2022. [Online]. Available: <https://arxiv.org/abs/2201.11182>

- [12] T. Kim, M. Jang, and J. Kim, “A survey on simulation environments for reinforcement learning,” in *2021 18th International Conference on Ubiquitous Robots (UR)*, 2021, pp. 63–67. [Online]. Available: <https://doi.org/10.1109/UR52253.2021.9494694>
- [13] E. Coumans and Y. Bai, “Pybullet, a python module for physics simulation for games, robotics and machine learning,” <http://pybullet.org>, 2016–2021.
- [14] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033. [Online]. Available: <https://doi.org/10.1109/IROS.2012.6386109>
- [15] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. d. Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. T. J. Shen, and O. G. Younis, “Gymnasium,” Mar. 2023. [Online]. Available: <https://zenodo.org/record/8127025>
- [16] J. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. S. Santos, C. Dieffendahl, C. Horsch, R. Perez-Vicente *et al.*, “Pettingzoo: Gym for multi-agent reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 15 032–15 043, 2021.
- [17] R. de Lazcano, K. Andreas, J. J. Tai, S. R. Lee, and J. Terry, “Gymnasium robotics,” 2023. [Online]. Available: <http://github.com/Farama-Foundation/Gymnasium-Robotics>
- [18] O. Nachum, S. S. Gu, H. Lee, and S. Levine, “Data-efficient hierarchical reinforcement learning,” in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/e6384711491713d29bc63fc5eeb5ba4f-Paper.pdf
- [19] M. Hutsebaut-Buysse, K. Mets, and S. Latré, “Hierarchical reinforcement learning: A survey and open research challenges,” *Machine Learning and Knowledge Extraction*, vol. 4, no. 1, pp. 172–221, 2022. [Online]. Available: <https://www.mdpi.com/2504-4990/4/1/9>
- [20] P.-L. Bacon, J. Harb, and D. Precup, “The option-critic architecture,” 2016. [Online]. Available: <https://arxiv.org/abs/1609.05140>
- [21] D. Jain, A. Iscen, and K. Caluwaerts, “Hierarchical reinforcement learning for quadruped locomotion,” 2019. [Online]. Available: <https://arxiv.org/abs/1905.08926>
- [22] J. Clement, “Video game market revenue worldwide from 2019 to 2029,” <https://www.statista.com/statistics/1344668/revenue-video-game-worldwide/>, (accessed: 06.09.2024).

- [23] S. Pateria, B. Subagdja, A.-h. Tan, and C. Quek, “Hierarchical reinforcement learning: A comprehensive survey,” *ACM Comput. Surv.*, vol. 54, no. 5, jun 2021. [Online]. Available: <https://doi.org/10.1145/3453160>